CSE1242 – C Data Structures

Dr. Sanem Arslan Yılmaz

Overview

12.1 Introduction

12.2 Self-Referential Structures

12.3 Dynamic Memory Allocation

12.4 Linked Lists

- 12.4.1 Function `insert`
- 12.4.2 Function `delete`
- 12.4.3 Function `printList`

12.5 Stacks

- 12.5.1 Function `push`
- 12.5.2 Function `pop`
- 12.5.3 Applications of Stacks

12.6 Queues

- 12.6.1 Function `enqueue`
- 12.6.2 Function `dequeue`

© 2016 Pearson Education, Ltd. All rights reserved.

12.1 Introduction

- We've studied fixed-size data structures such as *arrays* and *`structs`*.
- This lecture introduces dynamic data structures with sizes that grow and shrink at execution time.
  - Linked lists are collections of data items "lined up in a row"—insertions and deletions are made *anywhere* in a linked list.
  - Stacks are important in compilers and operating systems—insertions and deletions are made *only at one end* of a stack—its top.

© 2016 Pearson Education, Ltd. All rights reserved.

12.1 Introduction (Cont.)

  - Queues represent waiting lines; insertions are made *only at the back* (also referred to as the tail) of a queue and deletions are made *only from the front* (also referred to as the head) of a queue.
- Each of these data structures has many other interesting applications.

© 2016 Pearson Education, Ltd. All rights reserved.

12.2 Self-Referential Structures

- Recall that a *self-referential structure* contains a pointer member that points to a structure of the *same* structure type.
- For example, the definition

```
struct node {
   int data;
   struct node *nextPtr;
};
```

  defines a type, `struct node`.
- A structure of type `struct node` has two members—integer member `data` and pointer member `nextPtr`.

© 2016 Pearson Education, Ltd. All rights reserved.

12.2 Self-Referential Structures (Cont.)

- **`nextPtr`**
  - points to a structure of type `struct node`
  - referred to as a link
    - Ties a node to another node.

- Self-referential structures can be *linked* together to form useful data structures such as lists, queues and stacks.

© 2016 Pearson Education, Ltd. All rights reserved.

12.2 Self-Referential Structures (Cont.)

- Figure 12.1 illustrates two self-referential structure objects linked together to form a list.
- A NULL pointer normally indicates the end of a data structure just as the null character indicates the end of a string.



Fig. 12.1 | Self-referential structures linked together.

© 2016 Pearson Education, Ltd. All rights reserved.

12.3 Dynamic Memory Allocation

- Dynamic Memory Allocation
  - the ability for a program to *obtain more memory space at execution time* to hold new nodes, and to *release space no longer needed*.

- Functions `malloc` and `free`, and operator `sizeof`, are essential to dynamic memory allocation.

© 2016 Pearson Education, Ltd. All rights reserved.

12.3 Dynamic Memory Allocation (Cont.)

- **`void * malloc (int size)`**
  - takes as an argument the number of bytes to be allocated
    - Use **`sizeof`** to determine the size of an object
  - returns a pointer of type `void *` (pointer to void)
    - `void *` pointer may be assigned to *any* pointer type.
  - Example:

```
newPtr = malloc(sizeof(struct node));
```

- The allocated memory is *not* initialized.

- If no memory is available, `malloc` returns `NULL`.

© 2016 Pearson Education, Ltd. All rights reserved.

12.3 Dynamic Memory Allocation (Cont.)

- **`void free (void *ptr)`**
  - *deallocates* memory allocated by `malloc`
  - Takes a pointer as an argument
  - Example:

```
free(newPtr);
```

© 2016 Pearson Education, Ltd. All rights reserved.



Common Programming Error 12.2

Not `free`ing dynamically allocated memory when it's no longer needed can cause the system to run out of memory prematurely. This is sometimes called a "memory leak."



Error-Prevention Tip 12.2

When memory that was dynamically allocated is no longer needed, use `free` to return the memory to the system immediately. Then set the pointer to `NULL` to eliminate the possibility that the program could refer to memory that's been reclaimed and which may have already been allocated for another purpose.



Common Programming Error 12.3

Freeing memory not allocated dynamically with `malloc` is an error.



Common Programming Error 12.4

Referring to memory that has been freed is an error that typically results in the program crashing.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4 Linked Lists

- Linked List
  - a linear collection of self-referential structures, called nodes,
  - connected by pointer links—hence, the term "linked" list.
  - accessed via a pointer to the first node of the list.
  - Subsequent nodes are accessed via the link pointer member stored in each node.
  - Link pointer in the last node of a list is set to `NULL` to mark the end of the list.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4 Linked Lists (Cont.)

- Data is stored in a linked list dynamically—each node is created as necessary.
- Lists of data can be stored in arrays, but linked lists provide several advantages.
- A linked list is appropriate when the number of data elements is *unpredictable*.
- Linked lists are dynamic, so the length of a list can increase or decrease as necessary.
- Linked lists can be maintained in sorted order by inserting each new element at the proper point in the list.

© 2016 Pearson Education, Ltd. All rights reserved.



Performance Tip 12.1

An array can be declared to contain more elements than the number of data items expected, but this can waste memory. Linked lists can provide better memory utilization in these situations.



Performance Tip 12.2

Insertion and deletion in a sorted array can be time consuming—all the elements following the inserted or deleted element must be shifted appropriately.



Performance Tip 12.3

The elements of an array are stored contiguously in memory. This allows immediate access to any array element because the address of any element can be calculated directly based on its position relative to the beginning of the array. Linked lists do not afford such immediate access to their elements.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4 Linked Lists (Cont.)

- Linked-list nodes are normally *not* stored contiguously in memory.
- Logically, however, the nodes of a linked list *appear* to be contiguous.
- Figure 12.2 illustrates a linked list with several nodes.

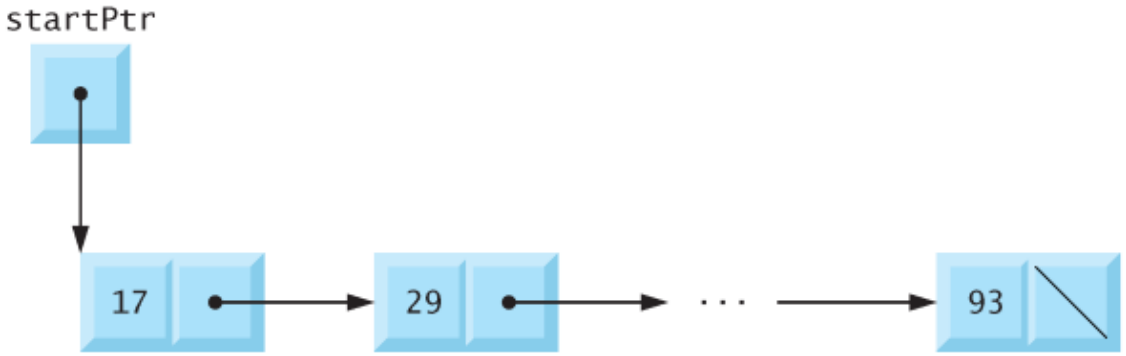


Fig. 12.2 | Linked-list graphical representation.

© 2016 Pearson Education, Ltd. All rights reserved.

Example

```
struct node {
    int data;
    struct node* next;
};
struct node* BuildOneTwoThree() {
  struct node* head = NULL;
  struct node* second = NULL;
  struct node* third = NULL;

  head = malloc(sizeof(struct node));
  second = malloc(sizeof(struct node));
  third = malloc(sizeof(struct node));

  head->data = 1;
  head->next = second;
  second->data = 2;
  second->next = third;
  third->data = 3;
  third->next = NULL;
  // At this point, the linked list referenced by "head"
  // matches the list in the drawing.
  return head;
}
```

© 2016 Pearson Education, Ltd. All rights reserved.

12.4 Linked Lists (Cont.)

- Example:
  - Figure 12.3 (output shown in Fig. 12.4) manipulates a list of characters.
  - You can insert a character in the list in alphabetical order (function `insert`) or to delete a character from the list (function `delete`).

© 2016 Pearson Education, Ltd. All rights reserved.

```c
// Fig. 12.3: fig12_03.c
// Inserting and deleting nodes in a list
#include <stdio.h>
#include <stdlib.h>

// self-referential structure
struct listNode {
   char data; // each listNode contains a character
   struct listNode *nextPtr; // pointer to next node
};

typedef struct listNode ListNode; // synonym for struct listNode
typedef ListNode *ListNodePtr; // synonym for ListNode*

// prototypes
void insert(ListNodePtr *sPtr, char value);
char delete(ListNodePtr *sPtr, char value);
int isEmpty(ListNodePtr sPtr);
void printList(ListNodePtr currentPtr);
void instructions(void);

int main(void)
{
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 1 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
24      ListNodePtr startPtr = NULL; // initially there are no nodes
25      char item; // char entered by user
26
27      instructions(); // display the menu
28      printf("%s", "? ");
29      unsigned int choice; // user's choice
30      scanf("%u", &choice);
31
32      // loop while user does not choose 3
33      while (choice != 3) {
34
35         switch (choice) {
36            case 1:
37               printf("%s", "Enter a character: ");
38               scanf("\n%c", &item);
39               insert(&startPtr, item); // insert item in list
40               printList(startPtr);
41               break;
42            case 2: // delete an element
43               // if list is not empty
44               if (!isEmpty(startPtr)) {
45                  printf("%s", "Enter character to be deleted: ");
46                  scanf("\n%c", &item);
47
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 2 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
48                   // if character is found, remove it
49                   if (delete(&startPtr, item)) { // remove item
50                      printf("%c deleted.\n", item);
51                      printList(startPtr);
52                   }
53                   else {
54                      printf("%c not found.\n\n", item);
55                   }
56                }
57                else {
58                   puts("List is empty.\n");
59                }
60
61                break;
62             default:
63                puts("Invalid choice.\n");
64                instructions();
65                break;
66          }
67
68          printf("%s", "? ");
69          scanf("%u", &choice);
70       }
71
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 3 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
72      puts("End of run.");
73   }
74
75   // display program instructions to user
76   void instructions(void)
77   {
78      puts("Enter your choice:\n"
79         "   1 to insert an element into the list.\n"
80         "   2 to delete an element from the list.\n"
81         "   3 to end.");
82   }
83
84   // insert a new value into the list in sorted order
85   void insert(ListNodePtr *sPtr, char value)
86   {
87      ListNodePtr newPtr = malloc(sizeof(ListNode)); // create node
88
89      if (newPtr != NULL) { // is space available?
90         newPtr->data = value; // place value in node
91         newPtr->nextPtr = NULL; // node does not link to another node
92
93         ListNodePtr previousPtr = NULL;
94         ListNodePtr currentPtr = *sPtr;
95
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 4 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
96          // loop to find the correct location in the list
97          while (currentPtr != NULL && value > currentPtr->data) {
98             previousPtr = currentPtr; // walk to ...
99             currentPtr = currentPtr->nextPtr; // ... next node
100         }
101
102         // insert new node at beginning of list
103         if (previousPtr == NULL) {
104            newPtr->nextPtr = *sPtr;
105            *sPtr = newPtr;
106         }
107         else { // insert new node between previousPtr and currentPtr
108            previousPtr->nextPtr = newPtr;
109            newPtr->nextPtr = currentPtr;
110         }
111      }
112      else {
113         printf("%c not inserted. No memory available.\n", value);
114      }
115   }
116
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 5 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
117 // delete a list element
118 char delete(ListNodePtr *sPtr, char value)
119 {
120    // delete first node if a match is found
121    if (value == (*sPtr)->data) {
122       ListNodePtr tempPtr = *sPtr; // hold onto node being removed
123       *sPtr = (*sPtr)->nextPtr; // de-thread the node
124       free(tempPtr); // free the de-threaded node
125       return value;
126    }
127    else {
128       ListNodePtr previousPtr = *sPtr;
129       ListNodePtr currentPtr = (*sPtr)->nextPtr;
130
131       // loop to find the correct location in the list
132       while (currentPtr != NULL && currentPtr->data != value) {
133          previousPtr = currentPtr; // walk to ...
134          currentPtr = currentPtr->nextPtr; // ... next node
135       }
136
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 6 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
137         // delete node at currentPtr
138         if (currentPtr != NULL) {
139            ListNodePtr tempPtr = currentPtr;
140            previousPtr->nextPtr = currentPtr->nextPtr;
141            free(tempPtr);
142            return value;
143         }
144      }
145
146      return '\0';
147   }
148
149   // return 1 if the list is empty, 0 otherwise
150   int isEmpty(ListNodePtr sPtr)
151   {
152      return sPtr == NULL;
153   }
154
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 7 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
155 // print the list
156 void printList(ListNodePtr currentPtr)
157 {
158    // if list is empty
159    if (isEmpty(currentPtr)) {
160       puts("List is empty.\n");
161    }
162    else {
163       puts("The list is:");
164
165       // while not the end of the list
166       while (currentPtr != NULL) {
167          printf("%c --> ", currentPtr->data);
168          currentPtr = currentPtr->nextPtr;
169       }
170
171       puts("NULL\n");
172    }
173 }
```

Fig. 12.3 | Inserting and deleting nodes in a list. (Part 8 of 8.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
Enter your choice:
   1 to insert an element into the list.
   2 to delete an element from the list.
   3 to end.
? 1
Enter a character: B
The list is:
B --> NULL

? 1
Enter a character: A

The list is:
A --> B --> NULL

? 1
Enter a character: C
The list is:
A --> B --> C --> NULL

? 2
Enter character to be deleted: D
D not found.
```

Fig. 12.4 | Sample output for the program of Fig. 12.3. (Part 1 of 2.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
? 2
Enter character to be deleted: B
B deleted.
The list is:
A --> C --> NULL

? 2
Enter character to be deleted: C
C deleted.
The list is:
A --> NULL

? 2
Enter character to be deleted: A
A deleted.
List is empty.

? 4
Invalid choice.

Enter your choice:
   1 to insert an element into the list.
   2 to delete an element from the list.
   3 to end.
? 3
End of run.
```

Fig. 12.4 | Sample output for the program of Fig. 12.3. (Part 2 of 2.)

12.4 Linked Lists (Cont.)

- Characters are inserted in the list in *alphabetical order*.
- Function `insert` receives the address of the list and a character to be inserted.
- The list's address is necessary when a value is to be inserted at the *start* of the list.
- Providing the address enables the list (i.e., the pointer to the first node of the list) to be *modified* via a call by reference.
- Because the list itself is a pointer (to its first element), passing its address creates a pointer to a pointer (i.e., double indirection).
- This is a complex notion and requires careful programming.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4.1 Function `insert`

- Steps for inserting a character in the list (Fig. 12.5):
  - *Create a node:*
    - call `malloc`,
    - assign to `newPtr` the address of the allocated memory,
    - assign the character to be inserted to `newPtr->data`,
    - assign `NULL` to `newPtr->nextPtr`
  - Initialize `previousPtr` to `NULL`
  - Initialize `currentPtr` to `*sPtr`—the pointer to the start of the list.
    - These pointers store the locations of the node *preceding* the insertion point and the node *after* the insertion point.
  - While `currentPtr` is not `NULL` and the value to be inserted is greater than `currentPtr->data`,
    - assign `currentPtr` to `previousPtr`
    - advance `currentPtr` to the next node in the list
    - This locates the insertion point for the value.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4.1 Function `insert` (Cont.)

- If `previousPtr` is `NULL`,
  - insert the new node as the first node in the list.
  - Assign `*sPtr` to `newPtr->nextPtr` (the new node link points to the former first node)
  - assign `newPtr` to `*sPtr` (`*sPtr` points to the new node).

- Otherwise, if `previousPtr` is not `NULL`,
  - the new node is inserted in place.
  - Assign `newPtr` to `previousPtr->nextPtr` (the *previous* node points to the new node)
  - assign `currentPtr` to `newPtr->nextPtr` (the *new* node link points to the *current* node).

© 2016 Pearson Education, Ltd. All rights reserved.

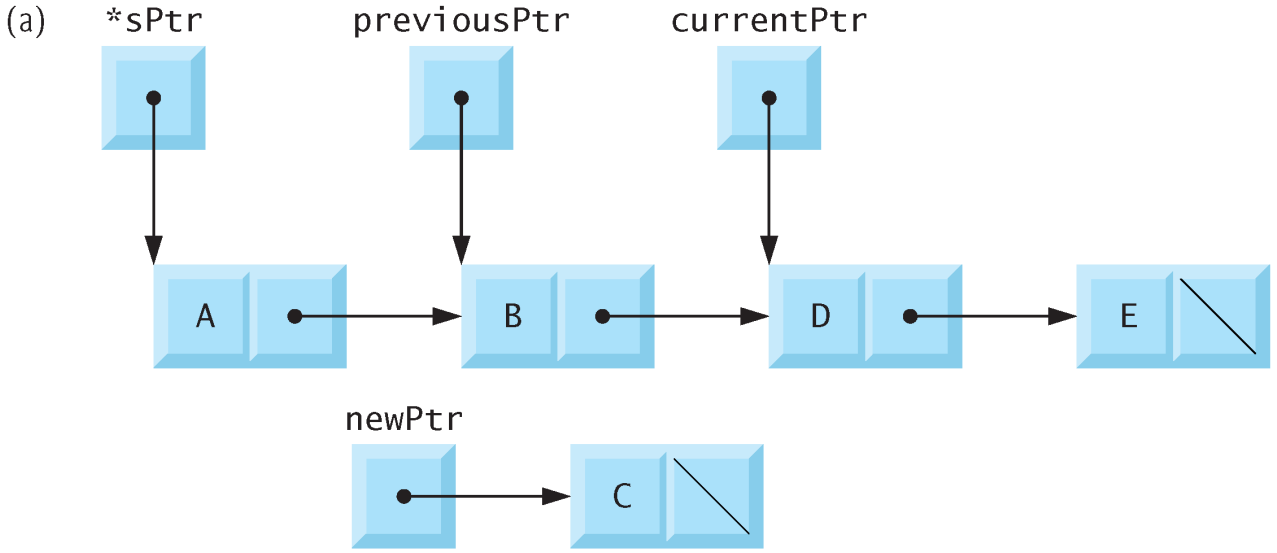


Fig. 12.5 | Inserting a node in order in a list.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4.2 Function `delete`

- Function `delete` receives the address of the pointer to the start of the list and a character to be deleted.

- Steps for deleting a character from the list:
  - If the character to be deleted matches the character in the first node of the list,
    - assign `*sPtr` to `tempPtr` (`tempPtr` will be used to `free` the unneeded memory),
    - assign `(*sPtr)->nextPtr` to `*sPtr` (`*sPtr` now points to the second node in the list),
    - `free` the memory pointed to by `tempPtr`,
    - return the character that was deleted.
  - Otherwise, initialize `previousPtr` with `*sPtr` and initialize `currentPtr` with `(*sPtr)->nextPtr` to advance the second node.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4.2 Function `delete` (cont.)

- Steps for deleting a character from the list:
  - While `currentPtr` is not `NULL` and the value to be deleted is not equal to `currentPtr->data`,
    - assign `currentPtr` to `previousPtr`,
    - assign `currentPtr->nextPtr` to `currentPtr`.
    - This locates the character to be deleted if it's contained in the list.
  - If `currentPtr` is not `NULL`,
    - assign `currentPtr` to `tempPtr`,
    - assign `currentPtr->nextPtr` to `previousPtr->nextPtr`,
    - free the node pointed to by `tempPtr`, and return the character that was deleted from the list.
  - If `currentPtr` is `NULL`,
    - return the null character (`'\0'`) to signify that the character to be deleted was not found in the list.

© 2016 Pearson Education, Ltd. All rights reserved.

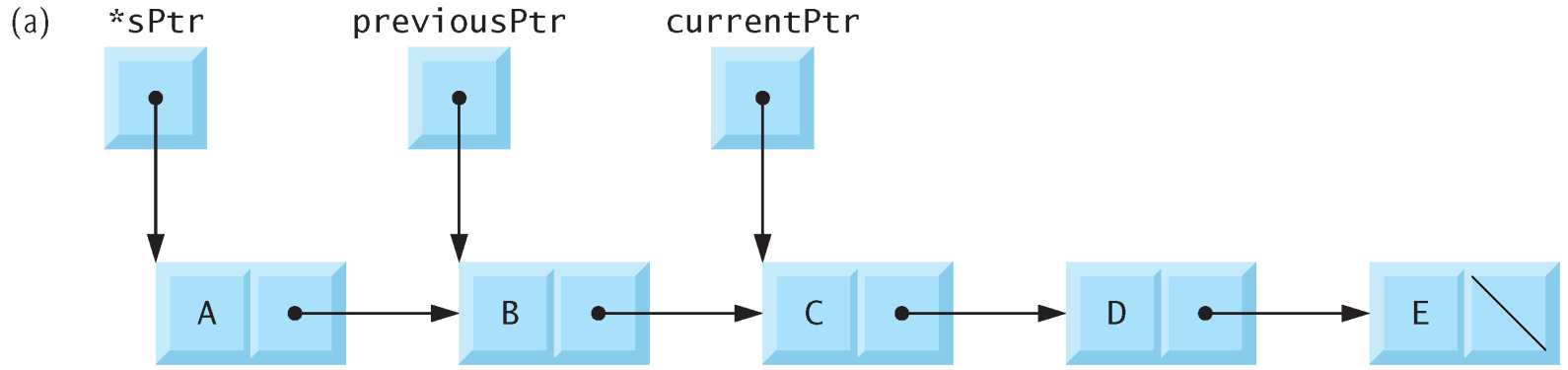


Fig. 12.6 | Deleting a node from a list.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4.3 Function `printList`

- Function `printList` receives a pointer to the start of the list as an argument and refers to the pointer as `currentPtr`.
- The function first determines whether the list is empty and, if so, prints "`List is empty.`" and terminates.
- Otherwise, it prints the data in the list
  - While `currentPtr` is not `NULL`,
    - the value of `currentPtr->data` is printed by the function,
    - `currentPtr->nextPtr` is assigned to `currentPtr` to advance to the next node.
- If the link in the last node of the list is not `NULL`, the printing algorithm will try to print *past the end of the list*, and an error will occur.
- The printing algorithm is identical for linked lists, stacks and queues.

© 2016 Pearson Education, Ltd. All rights reserved.

12.4 Linked Lists (Cont.)

- Types of linked lists:
  - Singly linked list
    - Begins with a pointer to the first node
    - Terminates with a null pointer
    - Only traversed in one direction
  - Circular, singly linked
    - Pointer in the last node points back to the first node
  - Doubly linked list
    - Two “start pointers” – first element and last element
    - Each node has a forward pointer and a backward pointer
    - Allows traversals both forwards and backwards
  - Circular, doubly linked list
    - Forward pointer of the last node points to the first node and backward pointer of the first node points to the last node

12.5 Stacks

- Stack
  - Can be implemented as a constrained version of a linked list.
  - New nodes can be added to a stack and removed from a stack *only* at the *top*.
  - Similar to a pile of dishes
  - Last-in, first-out (LIFO) data structure.
  - A stack is referenced via a pointer to the top element of the stack.
  - The link member in the last node of the stack is set to `NULL` to indicate the bottom of the stack.

© 2016 Pearson Education, Ltd. All rights reserved.

12.5 Stacks (Cont.)

- The difference between stacks and linked lists:
  - insertions and deletions may occur *anywhere* in a linked list,
  - but *only* at the *top* of a stack.

© 2016 Pearson Education, Ltd. All rights reserved.

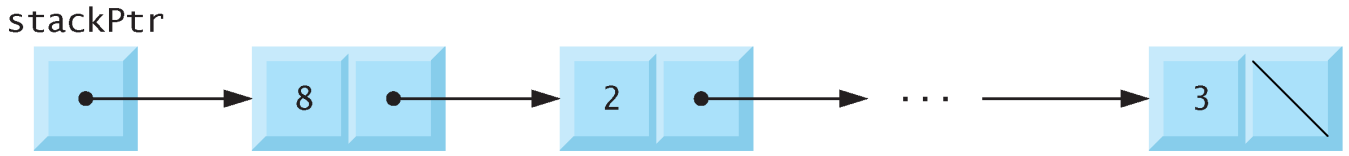


Fig. 12.7 | Stack graphical representation.

© 2016 Pearson Education, Ltd. All rights reserved.

12.5 Stacks (Cont.)

- The primary functions used to manipulate a stack are `push` and `pop`.
  - `push`
    - creates a new node and places it on top of the stack.
  - `pop`
    - *removes* a node from the *top* of the stack,
    - *frees* the memory that was allocated to the popped node
    - *returns the popped value*.

© 2016 Pearson Education, Ltd. All rights reserved.

12.5 Stacks (Cont.)

- Example:
  - Figure 12.8 (output shown in Fig. 12.9) implements a simple stack of integers.
  - The program provides three options:
    1) push a value onto the stack (function `push`),
    2) pop a value off the stack (function `pop`)
    3) terminate the program.

© 2016 Pearson Education, Ltd. All rights reserved.

```c
// Fig. 12.8: fig12_08.c
// A simple stack program
#include <stdio.h>
#include <stdlib.h>

// self-referential structure
struct stackNode {
   int data; // define data as an int
   struct stackNode *nextPtr; // stackNode pointer
};

typedef struct stackNode StackNode; // synonym for struct stackNode
typedef StackNode *StackNodePtr; // synonym for StackNode*

// prototypes
void push(StackNodePtr *topPtr, int info);
int pop(StackNodePtr *topPtr);
int isEmpty(StackNodePtr topPtr);
void printStack(StackNodePtr currentPtr);
void instructions(void);

```

Fig. 12.8 | A simple stack program. (Part 1 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
22  // function main begins program execution
23  int main(void)
24  {
25     StackNodePtr stackPtr = NULL; // points to stack top
26     int value; // int input by user
27
28     instructions(); // display the menu
29     printf("%s", "? ");
30     unsigned int choice; // user's menu choice
31     scanf("%u", &choice);
32
33     // while user does not enter 3
34     while (choice != 3) {
35
36        switch (choice) {
37           // push value onto stack
38           case 1:
39              printf("%s", "Enter an integer: ");
40              scanf("%d", &value);
41              push(&stackPtr, value);
42              printStack(stackPtr);
43              break;
```

Fig. 12.8 | A simple stack program. (Part 2 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
44            // pop value off stack
45            case 2:
46               // if stack is not empty
47               if (!isEmpty(stackPtr)) {
48                  printf("The popped value is %d.\n", pop(&stackPtr));
49               }
50
51               printStack(stackPtr);
52               break;
53            default:
54               puts("Invalid choice.\n");
55               instructions();
56               break;
57         }
58
59         printf("%s", "? ");
60         scanf("%u", &choice);
61      }
62
63      puts("End of run.");
64   }
65
```

Fig. 12.8 | A simple stack program. (Part 3 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
66  // display program instructions to user
67  void instructions(void)
68  {
69     puts("Enter choice:\n"
70        "1 to push a value on the stack\n"
71        "2 to pop a value off the stack\n"
72        "3 to end program");
73  }
74
75  // insert a node at the stack top
76  void push(StackNodePtr *topPtr, int info)
77  {
78     StackNodePtr newPtr = malloc(sizeof(StackNode));
79
80     // insert the node at stack top
81     if (newPtr != NULL) {
82        newPtr->data = info;
83        newPtr->nextPtr = *topPtr;
84        *topPtr = newPtr;
85     }
86     else { // no space available
87        printf("%d not inserted. No memory available.\n", info);
88     }
89  }
```

Fig. 12.8 | A simple stack program. (Part 4 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
90
91   // remove a node from the stack top
92   int pop(StackNodePtr *topPtr)
93   {
94      StackNodePtr tempPtr = *topPtr;
95      int popValue = (*topPtr)->data;
96      *topPtr = (*topPtr)->nextPtr;
97      free(tempPtr);
98      return popValue;
99   }
100
```

Fig. 12.8 | A simple stack program. (Part 5 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
101 // print the stack
102 void printStack(StackNodePtr currentPtr)
103 {
104    // if stack is empty
105    if (currentPtr == NULL) {
106       puts("The stack is empty.\n");
107    }
108    else {
109       puts("The stack is:");
110
111       // while not the end of the stack
112       while (currentPtr != NULL) {
113          printf("%d --> ", currentPtr->data);
114          currentPtr = currentPtr->nextPtr;
115       }
116
117       puts("NULL\n");
118    }
119 }
120
```

Fig. 12.8 | A simple stack program. (Part 6 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
121 // return 1 if the stack is empty, 0 otherwise
122 int isEmpty(StackNodePtr topPtr)
123 {
124    return topPtr == NULL;
125 }
```

Fig. 12.8 | A simple stack program. (Part 7 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
Enter choice:
1 to push a value on the stack
2 to pop a value off the stack
3 to end program
? 1
Enter an integer: 5
The stack is:
5 --> NULL

? 1
Enter an integer: 6
The stack is:
6 --> 5 --> NULL

? 1
Enter an integer: 4
The stack is:
4 --> 6 --> 5 --> NULL

? 2
The popped value is 4.
The stack is:
6 --> 5 --> NULL
```

Fig. 12.9 | Sample output from the program of Fig. 12.8. (Part 1 of 2.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
? 2
The popped value is 6.
The stack is:
5 --> NULL

? 2
The popped value is 5.
The stack is empty.

? 2
The stack is empty.

? 4
Invalid choice.

Enter choice:
1 to push a value on the stack
2 to pop a value off the stack
3 to end program
? 3
End of run.
```

Fig. 12.9 | Sample output from the program of Fig. 12.8. (Part 2 of 2.)

© 2016 Pearson Education, Ltd. All rights reserved.

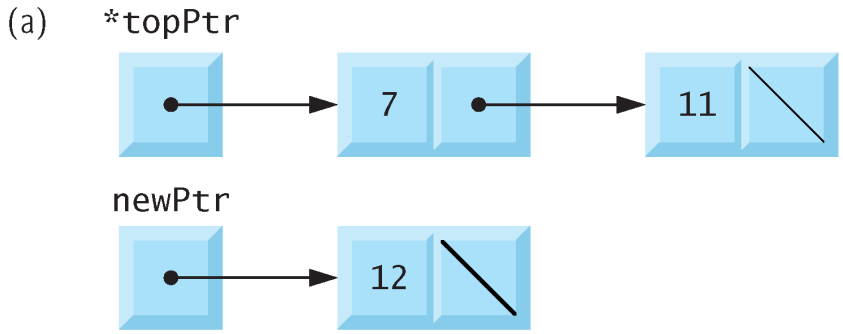


Fig. 12.10 | `push` operation.

© 2016 Pearson Education, Ltd. All rights reserved.

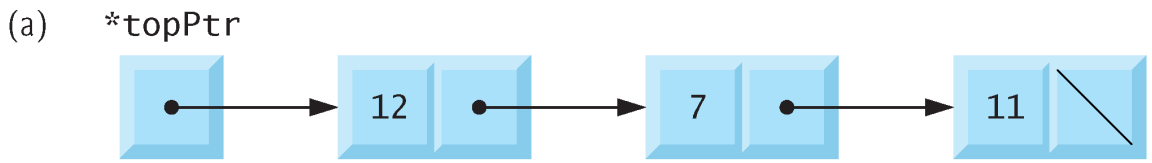


Fig. 12.11 | pop operation.

© 2016 Pearson Education, Ltd. All rights reserved.

12.5.3 Applications of Stacks

- Stacks have many interesting applications.
  - Function Call Stack
    - If a series of function calls occurs, the successive return values are pushed onto the stack in *last-in, first-out order* so that each function can return to its caller.
  - Stacks are used by compilers in the process of evaluating expressions and generating machine-language code.

© 2016 Pearson Education, Ltd. All rights reserved.

12.6 Queues

- Queues
  - similar to a checkout line in a grocery store
  - the *first* person in line is *serviced first*, and other customers enter the line only at the *end* and *wait* to be serviced.
  - nodes are removed *only* from the head of the queue
  - nodes are inserted *only* at the tail of the queue.
  - First-in, first-out (FIFO) data structure.
- The *insert* and *remove* operations
  - enqueue (insert) and dequeue (remove)

© 2016 Pearson Education, Ltd. All rights reserved.

12.6 Queues (Cont.)

- Queues have many applications in computer systems.
  - For computers that have only a single processor, only one user at a time may be serviced.
    - Entries for the other users are placed in a queue.
  - Queues are also used to support *print spooling*.
    - Many users may be generating outputs to be printed.
    - These are spooled to disk where they wait in a *queue* until the printer becomes available.

© 2016 Pearson Education, Ltd. All rights reserved.

12.6 Queues (Cont.)

- Figure 12.12 illustrates a queue with several nodes.
- Note the pointers to the head of the queue and the tail of the queue.

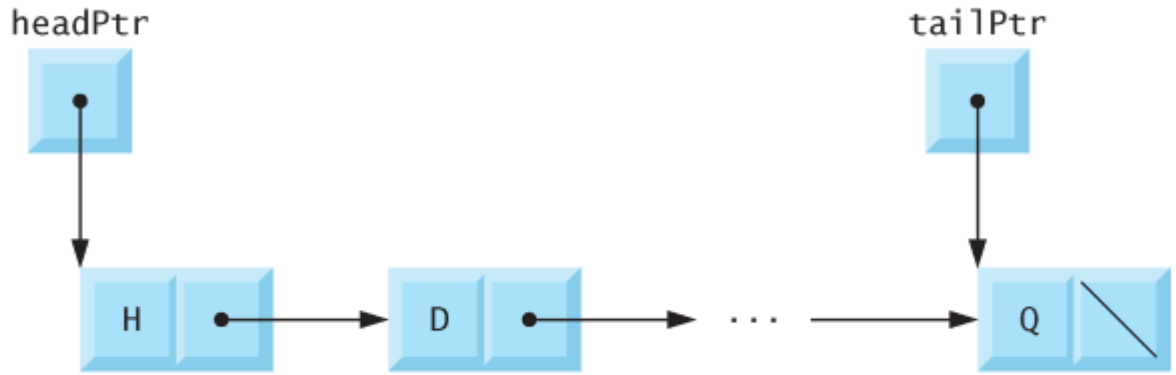


Fig. 12.12 | Queue graphical representation.

© 2016 Pearson Education, Ltd. All rights reserved.

```c
// Fig. 12.13: fig12_13.c
// Operating and maintaining a queue
#include <stdio.h>
#include <stdlib.h>

// self-referential structure
struct queueNode {
   char data; // define data as a char
   struct queueNode *nextPtr; // queueNode pointer
};

typedef struct queueNode QueueNode;
typedef QueueNode *QueueNodePtr;

// function prototypes
void printQueue(QueueNodePtr currentPtr);
int isEmpty(QueueNodePtr headPtr);
char dequeue(QueueNodePtr *headPtr, QueueNodePtr *tailPtr);
void enqueue(QueueNodePtr *headPtr, QueueNodePtr *tailPtr, char value);
void instructions(void);

// function main begins program execution
int main(void)
{
```

Fig. 12.13 | Operating and maintaining a queue. (Part 1 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
25      QueueNodePtr headPtr = NULL; // initialize headPtr
26      QueueNodePtr tailPtr = NULL; // initialize tailPtr
27      char item; // char input by user
28
29      instructions(); // display the menu
30      printf("%s", "? ");
31      unsigned int choice; // user's menu choice
32      scanf("%u", &choice);
33
34      // while user does not enter 3
35      while (choice != 3) {
36
37         switch(choice) {
38            // enqueue value
39            case 1:
40               printf("%s", "Enter a character: ");
41               scanf("\n%c", &item);
42               enqueue(&headPtr, &tailPtr, item);
43               printQueue(headPtr);
44               break;
```

Fig. 12.13 | Operating and maintaining a queue. (Part 2 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
45            // dequeue value
46            case 2:
47               // if queue is not empty
48               if (!isEmpty(headPtr)) {
49                  item = dequeue(&headPtr, &tailPtr);
50                  printf("%c has been dequeued.\n", item);
51               }
52
53               printQueue(headPtr);
54               break;
55            default:
56               puts("Invalid choice.\n");
57               instructions();
58               break;
59         }
60
61         printf("%s", "? ");
62         scanf("%u", &choice);
63      }
64
65      puts("End of run.");
66   }
67
```

Fig. 12.13 | Operating and maintaining a queue. (Part 3 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
68  // display program instructions to user
69  void instructions(void)
70  {
71     printf ("Enter your choice:\n"
72             "   1 to add an item to the queue\n"
73             "   2 to remove an item from the queue\n"
74             "   3 to end\n");
75  }
76
```

Fig. 12.13 | Operating and maintaining a queue. (Part 4 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
77  // insert a node at queue tail
78  void enqueue(QueueNodePtr *headPtr, QueueNodePtr *tailPtr, char value)
79  {
80     QueueNodePtr newPtr = malloc(sizeof(QueueNode));
81
82     if (newPtr != NULL) { // is space available?
83        newPtr->data = value;
84        newPtr->nextPtr = NULL;
85
86        // if empty, insert node at head
87        if (isEmpty(*headPtr)) {
88           *headPtr = newPtr;
89        }
90        else {
91           (*tailPtr)->nextPtr = newPtr;
92        }
93
94        *tailPtr = newPtr;
95     }
96     else {
97        printf("%c not inserted. No memory available.\n", value);
98     }
99  }
100
```

Fig. 12.13 | Operating and maintaining a queue. (Part 5 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
101 // remove node from queue head
102 char dequeue(QueueNodePtr *headPtr, QueueNodePtr *tailPtr)
103 {
104    char value = (*headPtr)->data;
105    QueueNodePtr tempPtr = *headPtr;
106    *headPtr = (*headPtr)->nextPtr;
107
108    // if queue is empty
109    if (*headPtr == NULL) {
110       *tailPtr = NULL;
111    }
112
113    free(tempPtr);
114    return value;
115 }
116
117 // return 1 if the queue is empty, 0 otherwise
118 int isEmpty(QueueNodePtr headPtr)
119 {
120    return headPtr == NULL;
121 }
122
```

Fig. 12.13 | Operating and maintaining a queue. (Part 6 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
123 // print the queue
124 void printQueue(QueueNodePtr currentPtr)
125 {
126    // if queue is empty
127    if (currentPtr == NULL) {
128       puts("Queue is empty.\n");
129    }
130    else {
131       puts("The queue is:");
132
133       // while not end of queue
134       while (currentPtr != NULL) {
135          printf("%c --> ", currentPtr->data);
136          currentPtr = currentPtr->nextPtr;
137       }
138
139       puts("NULL\n");
140    }
141 }
```

Fig. 12.13 | Operating and maintaining a queue. (Part 7 of 7.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
Enter your choice:
   1 to add an item to the queue
   2 to remove an item from the queue
   3 to end

? 1
Enter a character: A
The queue is:
A --> NULL

? 1
Enter a character: B
The queue is:
A --> B --> NULL

? 1
Enter a character: C
The queue is:
A --> B --> C --> NULL

? 2
A has been dequeued.
The queue is:
B --> C --> NULL
```

Fig. 12.14 | Sample output from the program in Fig. 12.13. (Part 1 of 2.)

© 2016 Pearson Education, Ltd. All rights reserved.

```
? 2
B has been dequeued.
The queue is:
C --> NULL

? 2
C has been dequeued.
Queue is empty.

? 2
Queue is empty.

? 4
Invalid choice.

Enter your choice:
   1 to add an item to the queue
   2 to remove an item from the queue
   3 to end
? 3
End of run.
```

Fig. 12.14 | Sample output from the program in Fig. 12.13. (Part 2 of 2.)

© 2016 Pearson Education, Ltd. All rights reserved.

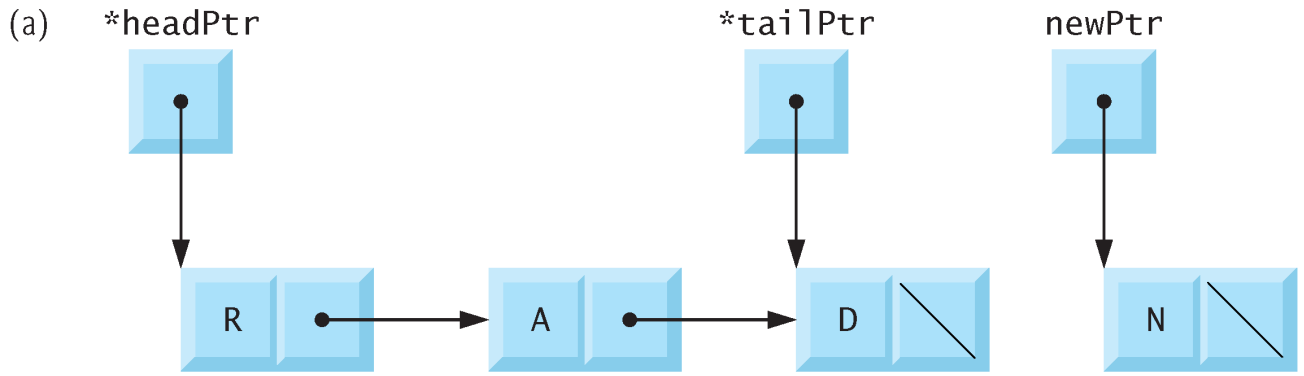


Fig. 12.15 | enqueue operation.

© 2016 Pearson Education, Ltd. All rights reserved.

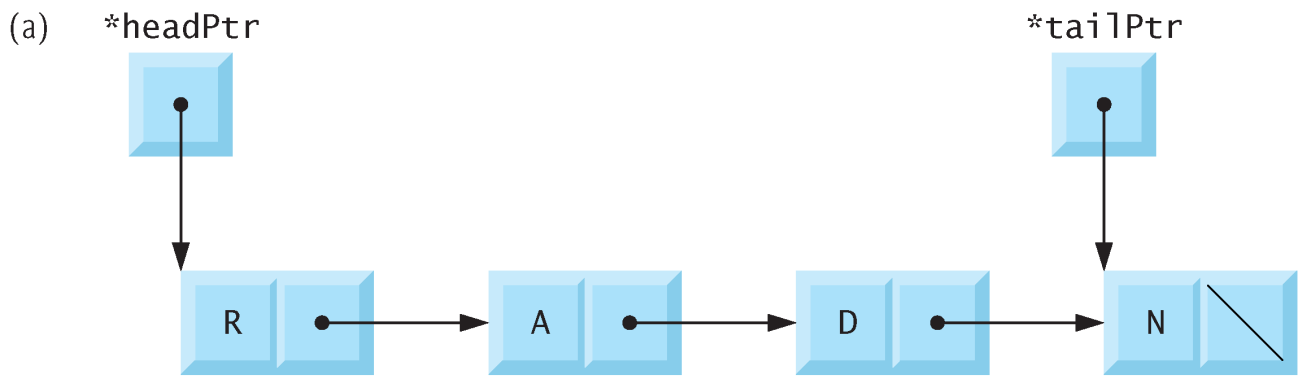


Fig. 12.16 | dequeue operation.

© 2016 Pearson Education, Ltd. All rights reserved.